



東南大學
SOUTHEAST UNIVERSITY

OPERATING SYSTEM CONCEPTS

.....

Chapter A0. Guide to Labworks

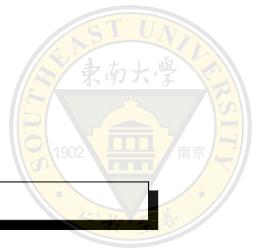
A/Prof. Kai Dong



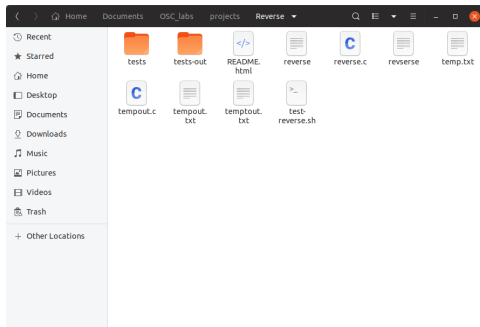
Contents

1. General Guide to Labworks
2. Guide to Debugging
3. Guide to Kernel Hacking

General Guide: Step #1



1 `cd projects/Reverse/`



Step #2

- Read carefully the corresponding “README.html” file.

A screenshot of a Mozilla Firefox browser window. The address bar shows the file path: file:///home/kai/Documents/OSC_labs/projects/Reverse/README.html. The page title is 'Introduction'. The content of the page is as follows:

This project is a simple warm-up to get you used to how this whole project thing will go. It also serves to get you into the mindset of a C programmer. You will write a simple program called `reverse`. This program should be invoked in one of the following ways:

```
prompt> ./reverse
prompt> ./reverse input.txt
prompt> ./reverse input.txt output.txt
```

The above line means the users typed in the name of the reversing program `reverse` (the `./` in front of it simply refers to the current working directory (called dot, referred to as `.`) and the slash (`/`) is a separator; thus, in this directory, look for a program named `reverse` and gave it either no command-line arguments, one command-line argument (an input file, `input.txt`), or two command-line arguments (an input file and an output file `output.txt`).

An input file might look like this:

```
hello
this
is
a file
```

The goal of the reversing program is to read in the data from the specified input file and reverse it; thus, the lines should be printed out in the reverse order of the input stream. Thus, for the aforementioned example, the output should be:

```
a file
is
this
hello
```

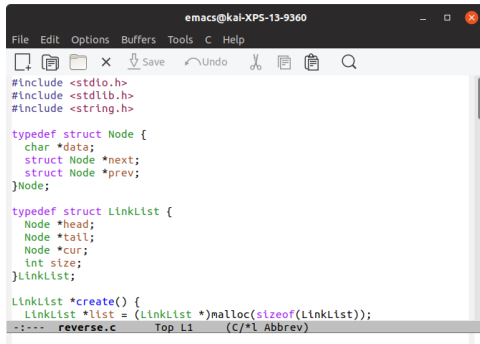
The different ways to invoke the file (as above) all correspond to slightly different ways of using this simple new Unix utility. For example, when invoked with two command-line arguments, the program should read from the input file the user supplies and write the reversed version of said file to the output file the user supplies.

When invoked with just one command-line argument, the user supplies the input file, but the file should be printed to the screen. In Unix-based systems, printing to the screen is the

Step #3

- Write your own reverse.c file.

1 emacs reverse.c



The screenshot shows the Emacs editor window titled 'emacs@kal-XPS-13-9360'. The menu bar includes 'File', 'Edit', 'Options', 'Buffers', 'Tools', 'C', and 'Help'. The toolbar contains icons for file operations and editing. The code in the buffer is as follows:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char *data;
    struct Node *next;
    struct Node *prev;
}Node;

typedef struct LinkList {
    Node *head;
    Node *tail;
    Node *cur;
    int size;
}LinkList;

LinkList *create() {
    LinkList *list = (LinkList *)malloc(sizeof(LinkList));
    --- reverse.c      Top L1      (C/*l Abbrev)
```



How to Use emacs

1. Type/edit as you wish.
2. Two commands you have to know (C is for Ctrl):

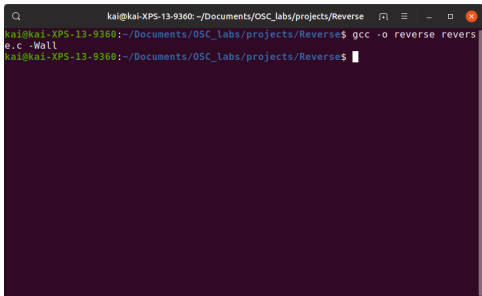
C-x C-s	保存文本
C-x C-c	退出 Emacs
3. Check the other commands by yourself if needed.

Step #4

- Compile it.

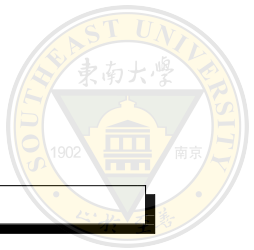
```
1 gcc -o reverse reverse.c -Wall
```

- If you are lucky:



```
kai@kai-XPS-13-9360: ~/Documents/OSC_labs/projects/Reverse
kai@kai-XPS-13-9360:~/Documents/OSC_labs/projects/Reverse$ gcc -o reverse reverse.c -Wall
kai@kai-XPS-13-9360:~/Documents/OSC_labs/projects/Reverse$
```

- Otherwise, debug it.

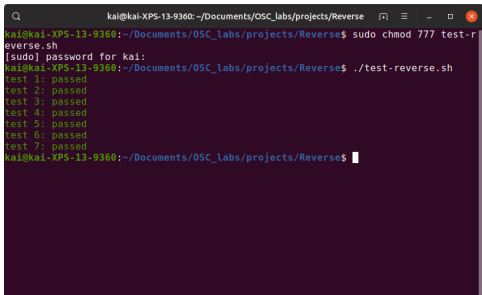


Step #5

- Run/test it.

```
1 sudo chmod 777 test -reverse.sh
2 ./test -reverse.sh
```

- If you are lucky:

A terminal window with a dark purple background. The title bar shows the path ~/Documents/OSC_labs/projects/Reverse. The user 'kai' is logged in on machine 'kai-XPS-13-9360'. The terminal shows the execution of 'sudo chmod 777 test-r', a password prompt, and then './test-reverse.sh'. The script outputs seven 'test X: passed' messages. The prompt returns to the shell.

```
kai@kai-XPS-13-9360: ~/Documents/OSC_labs/projects/Reverse
kai@kai-XPS-13-9360:~/Documents/OSC_labs/projects/Reverse$ sudo chmod 777 test-r
reverse.sh
[sudo] password for kai:
kai@kai-XPS-13-9360:~/Documents/OSC_labs/projects/Reverse$ ./test-reverse.sh
test 1: passed
test 2: passed
test 3: passed
test 4: passed
test 5: passed
test 6: passed
test 7: passed
kai@kai-XPS-13-9360:~/Documents/OSC_labs/projects/Reverse$
```

- Otherwise, debug it.



Contents



1. General Guide to Labworks
2. Guide to Debugging
3. Guide to Kernel Hacking

Guide to Debugging



This small guide can be very important!

- If you got errors/warnings when you compile your code, this guide is not at all helpful.
- If you pass the compiler check, but fail in test #*n*.
 1. Check the files "n.err", "n.out", "n.rc" in the folder named "/tests-out", this is what you put to stderr, stdout, and the return value.
 2. Compare to those files in "/tests", to see what they should be.
 3. If you still have difficulties, check "n.desc", "n.run" in "/texts" to see what test is actually performed in the testscript.

Contents



1. General Guide to Labworks
2. Guide to Debugging
3. Guide to Kernel Hacking



Guide to Kernel Hacking

Never hesitate to kernel hacking.

- Follow the Tips in "README.html" in "/Xv6-Syscall".
 - 30 ~ 60 mins for downloading.
- Read carefully "background.html" in "/Xv6-Syscall"
 - **This project is indeed SIMPLE!** If you are doing a lot of labor working — you are probably doing something wrong.
- Follow the Running Tests section in "README.html" in "/Xv6-Syscall".
- You might fail for test #2.
 - No worry, this is designed to. You can fix it in the next labwork.